

DashyDashboard

Server Setup Guide

Everything you do **on the server** — nothing else

June 2, 2026

Contents

1	What this guide covers	2
2	Step 1 — Install the ASP.NET Core Hosting Bundle	2
3	Step 2 — Download the App from GitHub	2
4	Step 3 — Create the Database Tables	3
5	Step 4 — Configure IIS	3
5.1	4.1 — Create an Application Pool	3
5.2	4.2 — Create the Website	4
5.3	4.3 — Enable Windows Authentication	4
5.4	4.4 — Tell the App to Use Production Settings	4
6	Step 5 — Verify It Works	5
7	Adding the First User	6
8	Database Hierarchy Reference	6

1 What this guide covers

This guide covers **only what you do on the server** (CLIPVWBPOD82 or whichever machine runs IIS and SQL Server). The laptop build is already done — the `upload/app/` folder in the GitHub repository is the finished, ready-to-deploy package.

Summary of steps

1. Install the ASP.NET Core runtime on the server (one-time, official Microsoft installer)
2. Download the app from GitHub
3. Create the database tables (SSMS — clicks only)
4. Configure IIS (IIS Manager — clicks only)
5. Verify it works

2 Step 1 — Install the ASP.NET Core Hosting Bundle

This is the **only installer you need to run**. It is an official Microsoft package, the same kind of thing as installing Office or Visual Studio.

1. On the server, open a browser and go to:
<https://dotnet.microsoft.com/en-us/download/dotnet/7.0>
2. Under **ASP.NET Core Runtime 7.x**, click **Hosting Bundle** (not SDK, not Runtime).
3. Download the **.exe** file.
4. Double-click it and click through the installer (Next, Next, Install). No options to change.
5. After it finishes, **open Command Prompt and type `iisreset`** then press Enter. This restarts IIS so it picks up the new runtime. (This is the only time you touch Command Prompt.)

Why: IIS cannot run ASP.NET Core applications without this. If it is already installed, skip this step — the installer will tell you.

3 Step 2 — Download the App from GitHub

1. On the server, open a browser and go to your GitHub repository.
2. Click the green **Code** button, then click **Download ZIP**.
3. Save the ZIP to a temporary location, e.g. the Desktop.
4. Right-click the ZIP → **Extract All...**

5. Navigate inside the extracted folder to:

```
upload\app\
```

6. Copy **the contents of** `upload\app\` (not the folder itself, everything inside it) to:

```
C:\inetpub\wwwroot\DashyDashboard\
```

Create the `DashyDashboard` folder first if it does not exist.

7. Also copy `upload\setup.sql` somewhere easy to find, e.g. the Desktop.

After this step, `C:\inetpub\wwwroot\DashyDashboard\` should contain `web.config`, `DashyDashboard.appsettings.Production.json`, and a `wwwroot\` folder.

4 Step 3 — Create the Database Tables

1. Open **SQL Server Management Studio (SSMS)**.
2. Connect to `CLIPVWBP0D82` using Windows Authentication.
3. In the left panel, expand the server, right-click **Databases** → **New Database...** Name it `ClientsAppAttestation`. Click OK. (Skip if the database already exists.)
4. Click **File** → **Open** → **File...** and open the `setup.sql` file you copied to the Desktop.
5. At the top of the query window, make sure the database dropdown shows `ClientsAppAttestation`.
6. Press **F5** (or click Execute).
7. You should see at the bottom:
`Schema created successfully. All 9 tables ready.`

If it says tables already exist: that is fine — the script drops old tables first. The drop order is safe (it removes child tables before parent tables). If it errors on the drop, you may have foreign-key constraints from an old schema — delete the tables manually in SSMS first (right-click each table → Delete), then run the script again.

SQL Server 2012 note: The script uses standard SQL that works on SQL Server 2012 SP4 and later. If you are on 2012 and the app fails to connect after this, open `C:\inetpub\wwwroot\DashyDashboard\appsettings.Production.json` in Notepad and change `Encrypt=True` to `Encrypt=False`.

5 Step 4 — Configure IIS

5.1 4.1 — Create an Application Pool

1. Open **IIS Manager** (Start → search “IIS” → Internet Information Services Manager).
2. In the left panel, click **Application Pools**.
3. In the right panel, click **Add Application Pool...**

4. Fill in:
 - **Name:** DashyDashboardPool
 - **.NET CLR version:** No Managed Code
 - **Managed pipeline mode:** Integrated
5. Click OK.

5.2 4.2 — Create the Website

1. In IIS Manager, right-click **Sites** → **Add Website...**
2. Fill in:
 - **Site name:** DashyDashboard
 - **Application pool:** click Select... and choose DashyDashboardPool
 - **Physical path:** C:\inetpub\wwwroot\DashyDashboard
 - **Port:** 80 (or any free port)
3. Click OK.

5.3 4.3 — Enable Windows Authentication

1. Click on the DashyDashboard site in the left panel.
2. In the middle panel, double-click **Authentication**.
3. **Anonymous Authentication** → right-click → **Disable**.
4. **Windows Authentication** → right-click → **Enable**.

If **Windows Authentication** is not in the list, it is not installed. Go to: Control Panel → Programs → Turn Windows features on or off → Internet Information Services → World Wide Web Services → Security → tick **Windows Authentication** → OK. Then repeat the steps above.

5.4 4.4 — Tell the App to Use Production Settings

The app has separate settings for Development and Production. On the server it must use Production (which has the correct database connection string). Do this through IIS:

1. Click on the DashyDashboard site.
2. Double-click **Configuration Editor** (if you do not see it, look under the “Management” section).
3. In the **Section** dropdown at the top, select: `system.webServer/aspNetCore`
4. Find the row **environmentVariables** and click the ... button next to it.
5. In the collection editor, click **Add**.
6. Set:

- **name:** ASPNETCORE_ENVIRONMENT
- **value:** Production

7. Click Close, then in the main Configuration Editor window click **Apply** (top-right).

Alternative (simpler): Open `C:\inetpub\wwwroot\DashyDashboard\web.config` in Notepad and add one line so the `aspNetCore` element looks like this:

```
<aspNetCore processPath="dotnet"
             arguments=".\\DashyDashboard.Api.dll"
             stdoutLogEnabled="false"
             hostingModel="InProcess">
  <environmentVariables>
    <environmentVariable name="ASPNETCORE_ENVIRONMENT"
                        value="Production" />
  </environmentVariables>
</aspNetCore>
```

Save the file. IIS picks up `web.config` changes automatically.

6 Step 5 — Verify It Works

1. In IIS Manager, click on the DashyDashboard site and then click **Browse *:80 (http)** in the right panel.
2. A browser should open and show the DashyDashboard login page.
3. Log in with a Windows domain account. (You will need to add that account to the `Users` table in SSMS first — see Section 7.)

If the site shows an error page

1. Open `web.config` in Notepad.
2. Change `stdoutLogEnabled="false"` to `stdoutLogEnabled="true"`.
3. Browse the site again to trigger the error.
4. Open the `logs\stdout` folder inside `C:\inetpub\wwwroot\DashyDashboard\` and read the log file — it will say exactly what went wrong.
5. Set `stdoutLogEnabled` back to `"false"` when done.

Common causes:

- **500.19 error:** Hosting Bundle not installed (Step 1).
- **Can't connect to SQL Server:** Check the connection string in `appsettings.Production.json`. For SQL Server 2012, try `Encrypt=False`.
- **Login page shows but login fails:** The Windows account is not in the `Users` table in SSMS.

7 Adding the First User

The database starts empty. Before anyone can log in, their Windows account must be in the Users table. In SSMS, run:

```
-- Replace values with real data
INSERT INTO dbo.Users
    (AssociateID, FirstName, LastName, UserName, Department)
VALUES
    ('A12345', 'Prakhar', 'Verma',
     'DOMAIN\prakharv', -- Windows login used by IIS
     'DTC Settlements');
```

To make someone an Admin so they can manage everyone:

```
INSERT INTO dbo.SuperUsers
    (AssociateID, RoleName, Department, AccessLevel, IsActive,
     CreatedBy)
VALUES
    ('A12345', 'Admin', 'DTC Settlements', 'Full', 'TRUE', '
    system');
```

8 Database Hierarchy Reference

SuperUsers (Admin / GFH / IFH)

```
|
+--> Department (e.g. "Operations")
    |
    +--> Clients WHERE Department = "Operations"
        |
        +--> ClientTools (tools per client)
            |
            +--> UsersToolAccess
                (who has which tool, from/to date)
                    |
                    +--> ToolCycleAttestation
                        (monthly: used it?)
```

Role summary

Role	Scope	Can do
Admin	Everything	All operations
GFH	One department	See all clients, add tools
IFH	One department	Same as GFH
Manager	Own team	See team progress, grant access
Associate	Own tools	Fill attestations, add remarks